

Aprendizaje Automático - Libro 1 - Clase 1 a 7

2. Clase 2

Clase N° 2: Adquisición de datos e Inspección y visualización de datos

Contenido: Adquisición de datos con la librería PANDAS. Utilización y alcance de la librería de Python Matplotlib.

En la clase de hoy trabajaremos los siguientes temas:

- Toma de datos con PANDAS:
 - Formato CSV
 - Desde Excel
 - Formato json
 - Formato html
 - Base de datos – SQLite3
 - Lenguaje XML
- Gráficos de línea.
- Gráficos de dispersión.
- Polinomios.
- Mapa de colores.

1. Presentación:

Bienvenidos a la clase N° 2 de Aprendizaje Automático.

En esta clase desarrollaremos un aspecto fundamental para cualquier proyecto de Aprendizaje Automático: la **adquisición de datos**. Como bien sabemos, los datos son la base sobre la cual construimos modelos, obtenemos insights y tomamos decisiones informadas. En este sentido, la librería Pandas de Python se erige como una herramienta imprescindible que nos permite trabajar de manera eficiente y efectiva con conjuntos de datos.

La librería Pandas es un componente crucial en el proceso de desarrollo de modelos de Aprendizaje Automático por varias razones clave:

Manipulación y Preparación de Datos: Antes de que cualquier algoritmo de Aprendizaje Automático pueda ser aplicado, los datos deben ser preparados y transformados adecuadamente. Pandas ofrece una amplia gama de funcionalidades para cargar, limpiar, transformar y filtrar datos de diversas fuentes, lo que facilita enormemente el proceso de preprocesamiento de datos.

Estructuras de Datos Flexibles: Pandas introduce dos estructuras de datos fundamentales: el DataFrame y la Serie. Estas estructuras permiten representar datos de manera tabular y en series temporales, respectivamente. La versatilidad de estas estructuras nos otorga la capacidad de trabajar con datos heterogéneos y multidimensionales, características esenciales en Aprendizaje Automático.

Indexación y Selección Intuitiva: Acceder y manipular datos específicos es esencial para el análisis y la construcción de modelos. Pandas proporciona métodos de indexación y selección de datos que simplifican enormemente estas tareas, permitiéndonos centrarnos en los aspectos más relevantes de nuestros datos.

Integración con Otras Librerías: Pandas se integra sin problemas con otras librerías populares en el ecosistema de Aprendizaje Automático, como NumPy y Scikit-Learn. Esta integración facilita la transición de los datos preprocesados a las etapas de modelado y evaluación.

Análisis Exploratorio de Datos (EDA): Antes de construir un modelo, es crucial comprender los datos en profundidad. Pandas ofrece herramientas para realizar análisis exploratorios, desde resúmenes estadísticos hasta visualizaciones, lo que nos permite detectar patrones, anomalías y relaciones en los datos.

Además también nos adentramos en una de las fases cruciales en el proceso de Aprendizaje Automático: **la inspección y visualización de datos**. Exploraremos cómo la librería Matplotlib, una herramienta poderosa de visualización en Python, juega un papel fundamental al convertir datos en información visualmente comprensible. Entender el porqué y el cómo de Matplotlib es esencial para cualquier persona involucrada en la creación y evaluación de modelos de Aprendizaje Automático.

La visualización de datos es más que una mera presentación estética. Se trata de una herramienta crucial en el arsenal de un científico de datos y un practicante de Aprendizaje Automático por diversas razones:

Comprensión de los Datos: Los conjuntos de datos suelen ser complejos y ricos en información. Visualizarlos nos permite comprender la distribución, tendencias y patrones presentes, lo que es fundamental para tomar decisiones informadas sobre qué técnicas de modelado aplicar.

Detección de Anomalías y Errores: Las visualizaciones pueden resaltar datos atípicos o errores en los datos. Identificar estas anomalías es crucial para asegurarnos de que nuestros modelos sean precisos y confiables.

Selección de Características: En muchas ocasiones, es necesario seleccionar las características más relevantes para entrenar modelos efectivos. La visualización puede ayudarnos a identificar aquellas características que están más relacionadas con la variable objetivo.

Evaluación de Modelos: La visualización no se limita a la fase de preparación de datos. También es esencial en la evaluación de modelos, permitiéndonos comprender cómo se desempeñan en diferentes escenarios y condiciones.

Comunicación Efectiva: Al presentar resultados a colegas, jefes o clientes, las visualizaciones claras y concisas pueden comunicar conceptos complejos de manera más efectiva que los datos en bruto.

Matplotlib es una biblioteca de visualización altamente personalizable y flexible. Ofrece una amplia variedad de gráficos, desde histogramas y gráficos de dispersión hasta gráficos de líneas y de barras, y permite personalizar cada aspecto de la presentación. Algunas razones por las que Matplotlib es esencial en Aprendizaje Automático son:

Variedad de Gráficos: Matplotlib proporciona una amplia gama de tipos de gráficos, lo que permite representar datos de manera adecuada según el contexto.

Personalización Completa: Cada elemento del gráfico puede ser personalizado, desde colores y etiquetas hasta títulos y ejes. Esto permite adaptar la visualización para resaltar aspectos específicos.

Integración con Pandas: Matplotlib se integra perfectamente con Pandas, lo que permite crear visualizaciones directamente desde los DataFrames, agilizando el proceso.

Compatibilidad con Jupyter Notebook: Matplotlib es una herramienta ideal para crear visualizaciones en entornos interactivos como Jupyter Notebook, lo que facilita la exploración de datos y la presentación de resultados.

2. Desarrollo y Actividades:

TOMAR DATOS CON PANDAS

Importar datos con pandas es muy simple, a continuación analizaremos diferentes fuentes de procedencia:

Formato CSV

Para tomar datos de un archivo csv, podemos partir de una hoja de Excel que posea una tabla como la siguiente:

id	Nombre	Genero
1	Juan	m
2	Pedro	m
3	Anna	f
4	Julieta	f
5	Pablo	m
6	Ramón	m
7	Gaby	f

Es necesario que dejemos solo una hoja en el libro y lo guardemos con formato '**csv delimitado por comas**'

Nota: Cuando tenemos configurada la maquina en español la separación entre datos es tomada por punto y coma (;) en lugar de coma (,). En este caso podemos utilizar algún editor de texto como notepad++ y sustituir todos los punto y comas (;) por comas (,).

Si ahora abrimos una hoja de jupyter podemos importar el archivo mediante el método `read_csv` de `pandas`.

```
import pandas as pd
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1")
dat_csv
```

La salida nos retorna:

Out[1]:

	id	Nombre	Genero	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000
5	6	Pedro	m	10500
6	7	Gaby	f	11000
7	8	Cecilia	f	27000

Si queremos que en lugar de todos los datos, se restrinja la salida a las primeras cinco filas, podemos utilizar el método **`head()`**.

```
In [2]: dat_csv.head()
```

```
Out[2]:
```

	id	Nombre	Genero	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000

En los dos casos anteriores, aparecen los elementos de la primera fila como los nombres de las columnas de datos, si no quisiéramos este comportamiento podemos agregar el atributo **header = None**, de la siguiente manera:

```
In [3]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", header = None)
dat_csv
```

```
Out[3]:
```

	0	1	2	3
0	id	Nombre	Genero	Sueldo
1	1	Juan	m	10000
2	2	Ramón	m	20000
3	3	Anna	f	10500
4	4	Julieta	f	13000
5	5	Pablo	m	30000
6	6	Pedro	m	10500
7	7	Gaby	f	11000
8	8	Cecilia	f	27000

Si quisiéramos que tomar una determinada línea como cabecera, debemos asignarle al parámetro header el número de fila correspondiente, en el siguiente ejemplo se toma la **línea 3** como cabecera:

```
In [4]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", header = 3)
dat_csv
```

```
Out[4]:
```

	3	Anna	f	10500
0	4	Julieta	f	13000
1	5	Pablo	m	30000
2	6	Pedro	m	10500
3	7	Gaby	f	11000
4	8	Cecilia	f	27000

Por defecto, **read_csv** asigna un índice numérico predeterminado que comienza con cero al leer los datos. Sin embargo, es posible alterar este comportamiento pasando el nombre de la columna que utilizaremos como índice. A continuación se dejara la columna **id** como índice de la tabla:

```
In [5]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", index_col='id')
dat_csv
```

Out[5]:

	Nombre	Genero	Sueldo
id			
1	Juan	m	10000
2	Ramón	m	20000
3	Anna	f	10500
4	Julieta	f	13000
5	Pablo	m	30000
6	Pedro	m	10500
7	Gaby	f	11000
8	Cecilia	f	27000

En el caso de que queramos restringir la tabla a algunas columnas específicas, podemos realizarlo con el parámetro **usecols** indicando en una lista las columnas seleccionadas.

```
In [6]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", usecols=['Nombre', 'Sueldo'])
dat_csv.head()
```

Out[6]:

	Nombre	Sueldo
0	Juan	10000
1	Ramón	20000
2	Anna	10500
3	Julieta	13000
4	Pablo	30000

Nota: Si alguna de las líneas se encuentra en blanco, podemos eliminar dichas líneas mediante el atributo **skip_blank_lines=False**

Si queremos eliminar una fila en particular, podemos utilizar el parámetro **skiprows**, en el siguiente caso se eliminan las filas 1 y 4:


```
In [7]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", skiprows = [1,4])
dat_csv
```

Out[7]:

	id	Nombre	Genero	Sueldo
0	2	Ramón	m	20000
1	3	Anna	f	10500
2	5	Pablo	m	30000
3	6	Pedro	m	10500
4	7	Gaby	f	11000
5	8	Cecilia	f	27000

Para presentar un número dado de filas desde el inicio, podemos utilizar el parámetro **nrows**, en el siguiente ejemplo se presentan las primeras tres filas de datos.

```
In [8]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1", nrows=3)
dat_csv
```

Out[8]:

	id	Nombre	Genero	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500

Para seleccionar un rango específico de filas podemos utilizar el método `query` especificando los límites aplicables sobre una determinada columna:

```
In [9]: dat_csv = pd.read_csv('datos/empleados.csv', encoding = "ISO-8859-1")
dat_csv.query('2 < id < 6')
```

Out[9]:

	id	Nombre	Genero	Sueldo
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000

Para tratar con los datos de una columna, podemos recuperarlos dentro de una lista mediante el uso de un **bucle for**, y luego si es necesario convertir la lista en una array como se muestra a continuación:


```
In [10]: import pandas as pd
import numpy as np
dat_csv = pd.read_csv('datos/datos.csv', encoding = "ISO-8859-1")
datos_x = dat_csv.nombre
datos_y = dat_csv.genero
x = []
y = []
for i in dat_csv.nombre:
    x.append(i)
for j in dat_csv.genero:
    y.append(j)
print(x)
print(y)

x_array = np.array(x)
print(type(x_array))
print(x_array)

['Juan', 'Pedro', 'Marcelo', 'Anna']
['m', 'm', 'm', 'f']
<class 'numpy.ndarray'>
['Juan' 'Pedro' 'Marcelo' 'Anna']
```

Desde Excel

En el caso de trabajar directamente con un archivo de Excel, podemos utilizar el método **read_excel()** de forma análoga a como lo hicimos anteriormente con **read_csv()**

```
In [11]: import pandas as pd
dat_excel = pd.read_excel('datos/empleados.xlsx')
dat_excel.head()
```

Out[11]:

	id	Nombre	Genero	Sueldo
0	1	Juan	m	10000.0
1	2	Ramón	m	20000.0
2	3	Anna	f	10500.0
3	4	Julieta	f	13000.0
4	5	Pablo	m	30000.0

Nota: El parámetro sheetname nos permite seleccionar el número de hoja, las cuales se comienzan a numerar desde cero.

Formato json

El formato JSON es un lenguaje de intercambios de datos como XML pero mucho más simple. Es un lenguaje muy utilizado en el envío de datos desde aplicaciones web y móviles realizadas con javascript. Por su similitud con el tipo de datos de objetos de javascript ha adquirido un papel muy importante en desarrolladores de esta comunidad. También es muy utilizado por los desarrolladores de videojuegos y animaciones para describir las partes del cuerpo de un avatar o darle formato a las texturas de un objeto.

El formato JSON para los que trabajamos con python sería muy similar asemejarse a una lista de diccionarios, o sea que podemos pensar en la siguiente estructura:

```
[
  {
    "name": "John",
    "age": 30,
    "gender": "male"
  },
  {
    "name": "Jane",
    "age": 25,
    "gender": "female"
  }
]
```

De hecho cuando transformamos un json a python es entendido exactamente así, como una lista de diccionarios. Consideremos el siguiente ejemplo:

```
json1.json
1  {
2      "nombre": "Ana",
3      "edad": 33,
4      "estado_civil": true,
5      "esposo": "Pablo",
6      "hijos": ["Cecilia", "Luis"],
7      "autos": [
8          {
9              "modelo": "Ford",
10             "color": "rojo"
11         },
12         {
13             "modelo": "Chevrolet",
14             "color": "azul"
15         }
16     ]
17 }
```

Pandas utiliza el método **read_json()** para importar un archivo con este formato como se muestra a continuación:

Formato html

Con pandas podemos importar un formato html desde la web o desde un archivo local, los datos los debemos guardar en tablas. Los datos tomados son los correspondientes a las filas <tr></tr> según se muestra a continuación:

```
In [12]: import pandas as pd
movies_json = pd.read_json('datos/json1.json')
movies_json.head()
```

Out[12]:

	nombre	edad	estado_civil	esposo	hijos	autos
0	Ana	33	True	Pablo	Cecilia	{'modelo': 'Ford', 'color': 'rojo'}
1	Ana	33	True	Pablo	Luis	{'modelo': 'Chevrolet', 'color': 'azul'}

Formato html

Con pandas podemos importar un formato html desde la web o desde un archivo local, los datos los debemos guardar en tablas. Los datos tomados son los correspondientes a las filas `<tr>` `</tr>` según se muestra a continuación:

```
index.html
1  <!Doctype html>
2  <html>
3    <head>
4      <title>
5        Mi sitio
6      </title>
7    </head>
8    <body>
9      <h1>Esta es una tabla</h1>
10     <div>
11       <table width="100%" border=1 cellpadding=1 cellspacing=2 >
12         <tr valign=top>
13           <td style="border-style: inset;">
14             <p><span class=rvts1>id</span></p>
15           </td>
16           <td style="border-style: inset;">
17             <p><span class=rvts1>Nombre</span></p>
18           </td>
19           <td style="border-style: inset;">
20             <p><span class=rvts1>Sexo</span></p>
21           </td>
22           <td style="border-style: inset;">
23             <p><span class=rvts1>Sueldo</span></p>
24           </td>
25         </tr>
26
27
28         <tr valign=top>
29           <td style="border-style: inset;">
30             <p><span class=rvts1>1</span></p>
31           </td>
32           <td style="border-style: inset;">
33             <p><span class=rvts1>Anna</span></p>
34           </td>
35           <td style="border-style: inset;">
36             <p><span class=rvts1>f</span></p>
37           </td>
38           <td style="border-style: inset;">
39             <p><span class=rvts1>10000</span></p>
40           </td>
41         </tr>
42       </table>
43     </div>
44   </body>
45 </html>
```

```
In [13]: import pandas as pd
pd.read_html('datos/index.html')
```

```
Out[13]: [ 0      1      2      3
           0  id  Nombre  Sexo  Sueldo
           1  1   Anna    f   10000]
```

Base de datos – SQLite3

También podemos trabajar con SQLite 3, el cual es un tipo de base de datos relacional como MySQL o MariaDB. Para trabajar con este tipo de bases de datos podemos descargar SQLITE Studio, el cual es un administrador gráfico de este tipo de bases de datos del siguiente sitio: <https://sqlitestudio.pl/>

La conexión requiere:

- Importar el módulo para trabajar con esta base de datos
- Declarar en el método **connect()** el nombre de la base y la ruta a la misma.
- Introducir la sentencia de búsqueda en lenguaje sql mediante el método **read_sql_query()**

```
In [14]: import pandas as pd
import sqlite3
conn = sqlite3.connect("datos/mibase.sqlite")
df = pd.read_sql_query("SELECT * FROM producto;", conn)
df.head()
```

```
Out[14]:
```

	id	nombre	sexo
0	1	Pedro	m
1	2	Anna	f
2	3	Celeste	f

Lenguaje XML

Xml es un lenguaje de etiquetas, es utilizado como lenguaje de transferencia de datos e incluso en algunas plataformas como android es utilizado para crear las vistas de las pantallas. Para ver como importar los datos partiremos del siguiente ejemplo:


```
1  <?xml version="1.0"?>
2  <datos>
3      <cliente nombre="Juan" >
4          <email>juan@gmail.com</email>
5          <telefono>011-111111111</telefono>
6          <direccion>
7              <calle>Tacuarí 342</calle>
8          </direccion>
9      </cliente>
10     <cliente nombre="Anna" >
11         <email>anna@gmail.com</email>
12     </cliente>
13     <cliente nombre="Pedro" >
14         <email>pedro@gmail.com</email>
15         <telefono>011-222222222</telefono>
16         <direccion>
17             <calle>Río de Janeiro 215</calle>
18         </direccion>
19     </cliente>
20     <cliente nombre="Gabriela" >
21         <email>gabriela@gmail.com</email>
22         <telefono>011-333333333</telefono>
23         <direccion>
24             <calle>Rivadavia 8345</calle>
25         </direccion>
26     </cliente>
27 </datos>
```

Para trabajar con los datos podemos utilizar la API XML de ElementTree, que es un procesador XML simple y ligero. Para leer y analizar un archivo XML, todo lo que necesitamos es llamar al método de análisis "**parse**" :

```
import xml.etree.cElementTree as et
parsed_xml = et.parse("datos/datos.xml")
```

El código anterior retorna un objeto ElementTree, y podemos usar el método "**iter ()**" para generar un iterador (para elementos XML específicos) o "**getroot ()**" para obtener el elemento raíz de este árbol, y luego iterar todos los elementos.

En el siguiente ejemplo de código se utiliza la función main para recorrer los valores del documento y darle formato


```
In [30]: import xml.etree.cElementTree as et
import pandas as pd

def obtenerValorDeNodo(node):
    return node.text if node is not None else None

def main():
    parsed_xml = et.parse("datos/datos.xml")
    dfcols = ['nombre', 'email', 'telefono', 'calle']
    df_xml = pd.DataFrame(columns=dfcols)

    for node in parsed_xml.getroot():
        nombre = node.attrib.get('nombre')
        email = node.find('email')
        telefono = node.find('telefono')
        calle = node.find('direccion/calle')

        df_xml = df_xml.append( pd.Series([nombre, obtenerValorDeNodo(email),
                                          obtenerValorDeNodo(telefono), obtenerValorDeNodo(calle)],
                                          index=dfcols), ignore_index=True)
    print(df_xml)

main()
```

	nombre	email	telefono	calle
0	Juan	juan@gmail.com	011-111111111	Tacuari 342
1	Anna	anna@gmail.com	None	None
2	Pedro	pedro@gmail.com	011-222222222	Río de Janeiro 215
3	Gabriela	gabriela@gmail.com	011-333333333	Rivadavia 8345

Matplotlib

Vamos a iniciar a trabajar con matplotlib aprendiendo a agregar comentarios y descripciones a los gráficos, para ello crearemos un gráfico con tres curvas en el dominio $x = [0, 2]$ y adicionamos 100 puntos equiespaciados dentro del dominio mediante:

```
import numpy as np
x = np.linspace(0, 2, 100)
x
```

Luego podemos crear las curvas utilizando el método plot() en donde pasamos como parámetros:

- Primero los puntos correspondientes a la coordenada en x.
- Segundo los puntos de la coordenada en y.

- Tercero el color de línea o el tipo de representación para los puntos de la línea, si en lugar de poner 'pink' ponemos '+' cada punto es representado por un signo de (+).
- Cuarto el atributo **label** con la descripción de la curva. Para que la descripción salga en pantalla debemos agregar el método **legend()**.

```
import matplotlib.pyplot as plt
x = np.linspace(0, 2, 100)
plt.plot(x, x, 'pink', label='lineal')
plt.plot(x, x**2, label='cuadratica')
plt.plot(x, x**3, label='cubica')

plt.legend()
```

Colores

Para los colores también tenemos otras opciones como:

- Dar el valor hexadecimal de forma análoga a como se hace en una página web dentro de código css. Los colores representan tres canales de colores:
 - Canal rojo #ff0000
 - Canal verde #00ff00
 - Canal azul #0000ff
- Se puede dar una abreviatura para el color, por ejemplo 'm' representa el color magenta.
- Se puede especificar el parámetro c con un valor de C0 al C9, lo cual agrega colores con buen contraste preestablecidos.
- Los colores pueden ser dados en canales de colores rgb en donde se dan tres parámetros entre 0 y 1 representando los canales rojo, verde y azul de la siguiente manera: c=(1,0.1,1)
- Los colores pueden ser dados en canales de colores rgba en donde se dan tres parámetros entre 0 y 1 representando los canales rojo, verde y azul y un cuarto parámetro que determina el nivel de opacidad, de la siguiente manera: c=(1,0.1,1,0.5)

Descripción de curva

La descripción de la curva puede posicionarse en la pantalla si tomamos la longitud de cada eje como si estuviera en el intervalo **(0,1)**, por lo que podemos en base a esto agregar dentro de **legend()** la ubicación según cada eje con el uso del parámetro **loc**. De esta forma podemos darle una posición específica dentro del gráfico de la siguiente forma: **plt.legend(loc=(0.5,0.7))**

Descripciones de título y ejes

Para agregar descripciones a los ejes y título utilizamos los métodos:

- xlabel() para agregar una descripción en el eje x.
- ylabel() para agregar una descripción en el eje y.
- title() para adicionar un título al gráfico.

```
plt.xlabel('Eje X')  
plt.ylabel('Eje Y')  
plt.title("Estructura de gráfico")
```

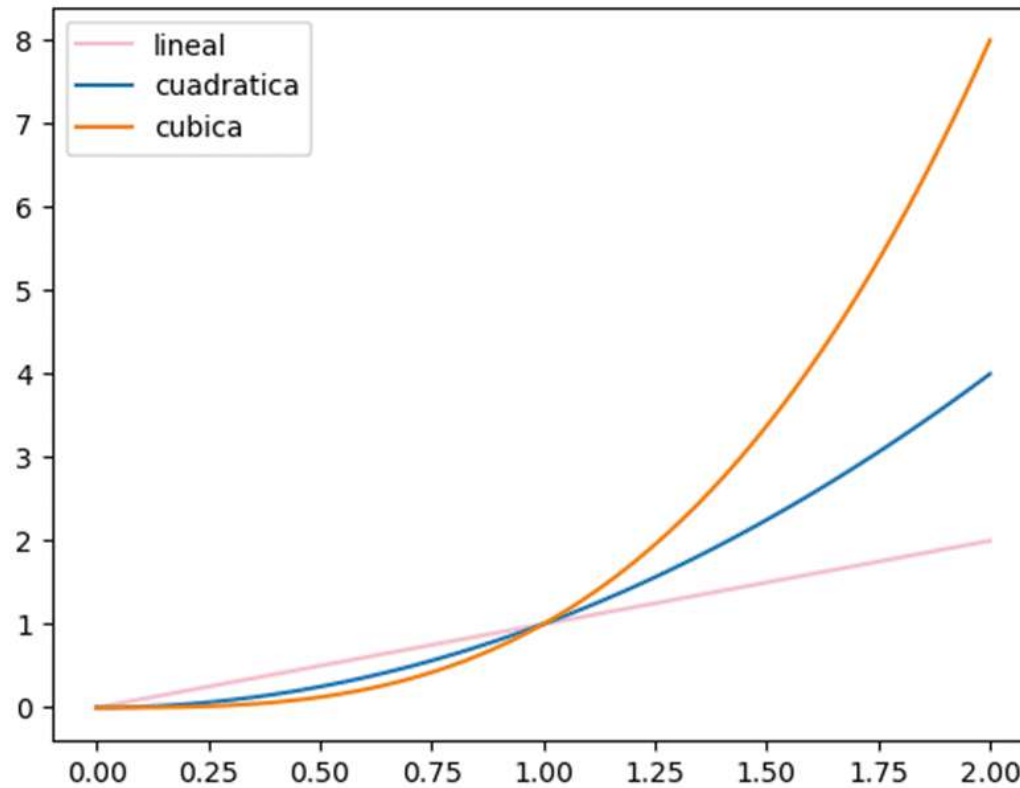
Límites en ejes

También podemos especificar los límites de la representación de datos en los ejes mediante:

- plt.ylim(valor mínimo, valor máximo) para el eje y.
- plt.xlim(valor mínimo, valor máximo) para el eje x.

```
plt.xlim(0,2)  
plt.ylim(0,8)
```

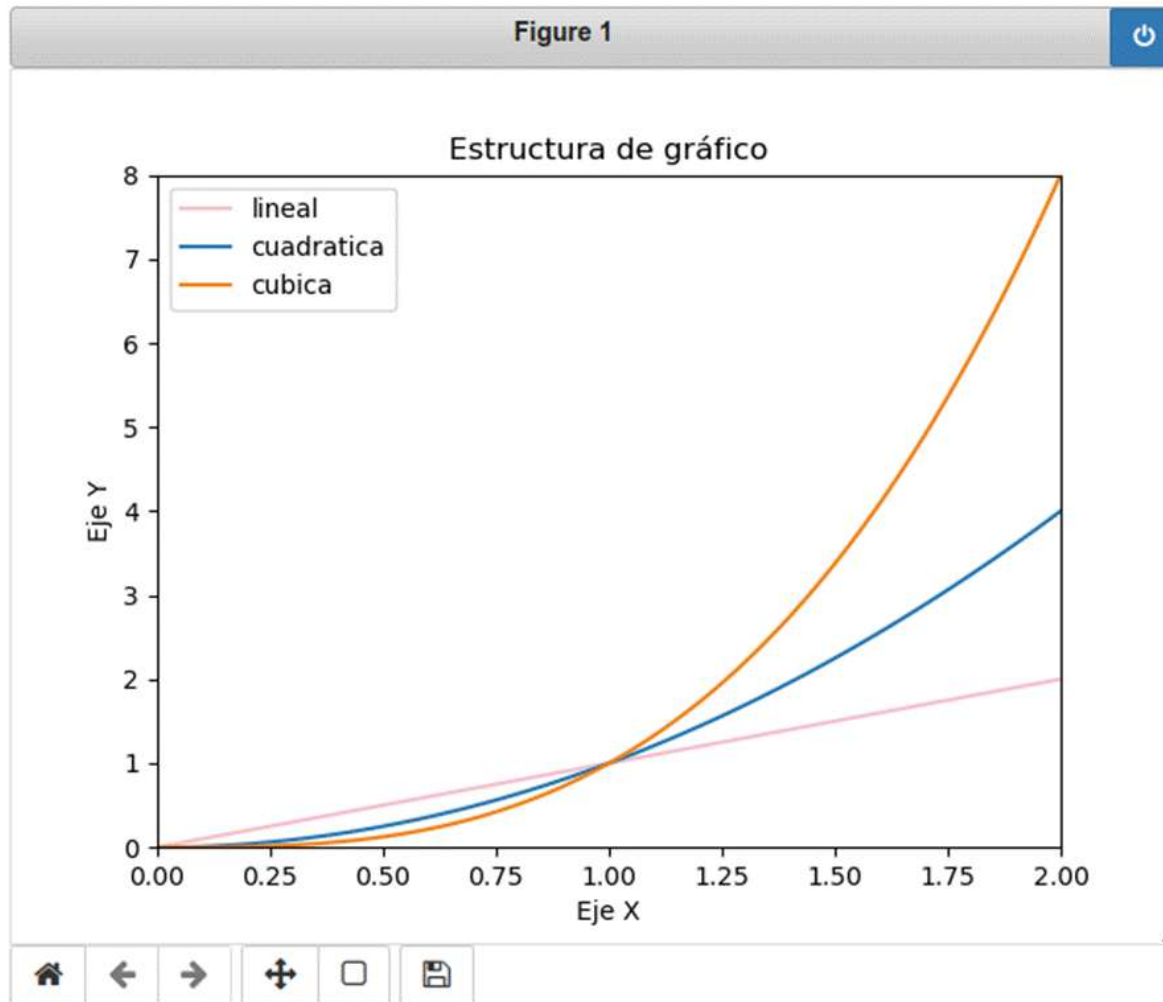
Si queremos adicionar una grilla lo podemos realizar mediante el método **grid()**. La representación visual nos queda:



Si al inicio de nuestro código agregamos la línea:

```
%matplotlib notebook
```

Nuestro gráfico se vuelve interactivo y aparecen herramientas de navegación de forma de poder hacer zoom en un área o desplazar el gráfico para posicionarnos en un punto específico, también aparece un tirador en el margen inferior derecho desde el cual podemos aumentar o disminuir el tamaño del gráfico:



ICONO	DESCRIPCIÓN
[1] Casa	Restablecer vista original
Flecha izquierda	Volver a la vista anterior
Flecha derecha	Ir a la siguiente vista

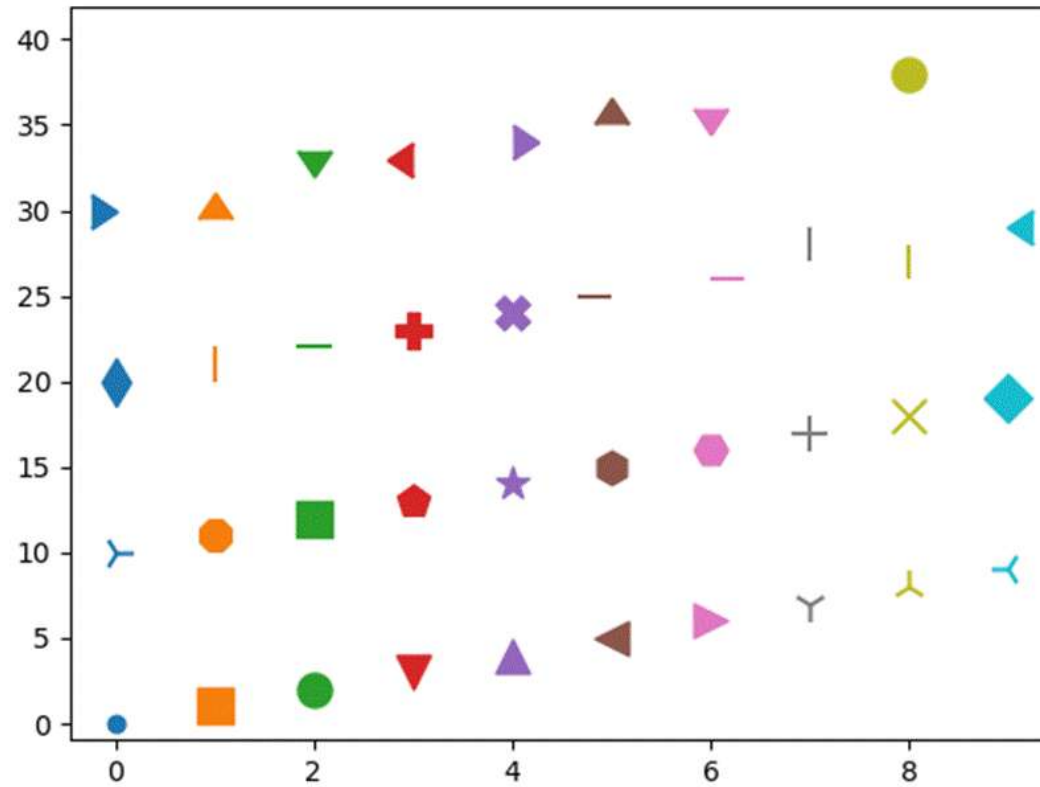
ICONO	DESCRIPCIÓN
[1] Casa	Restablecer vista original
Flecha en cuatro direcciones	Paneo manteniendo presionada la tecla izquierda del mouse y Zoom con la tecla de flecha derecha en la pantalla.
Rectángulo	Zoom de enmarcado.
Disquete	Guardar

Tipos de marcas

En lugar de obtener puntos en las dispersiones, podemos agregar distintos tipo de indicadores, podemos verlos todos al ejecutar el siguiente código:

```
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

for i,marker in enumerate(Line2D.markers):
    plt.scatter(i%10,i,marker=marker,s=150)
plt.show()
```

El tamaño del indicador lo puedo modificar con el parámetro **s**.

Estilos de línea

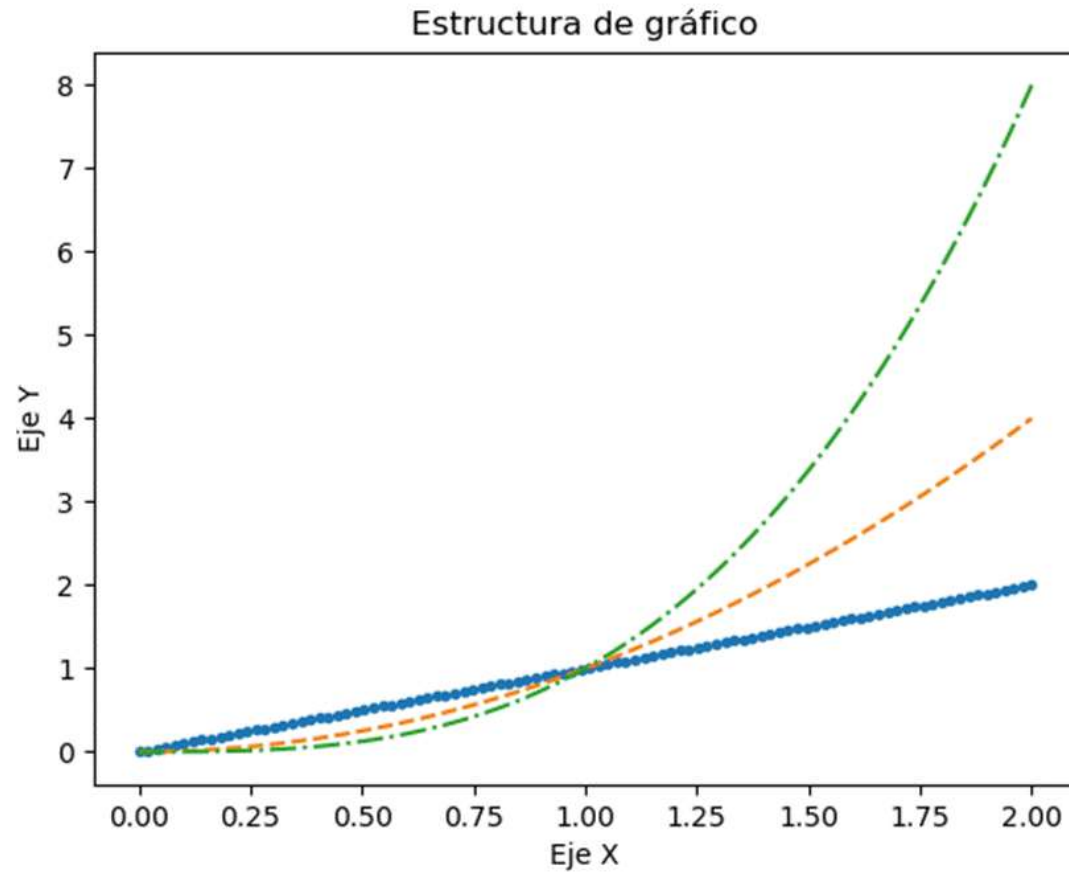
Podemos agregar diferentes tipos de trazado de línea utilizando los siguientes parámetros:

PARÁMETRO	TIPO DE LÍNEA
'-'	Línea llena
'--'	Línea discontinua
'-.'	Línea de raya punto

'.'	Línea de puntos grandes
'-.'	Línea de puntos

Veamos un ejemplo:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2, 100)
plt.plot(x, x, '.', label='.')
plt.plot(x, x**2, '--', label='--')
plt.plot(x, x**3, '-.', label='-.')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.title("Estructura de gráfico")
plt.show()
```



Gráficos de línea

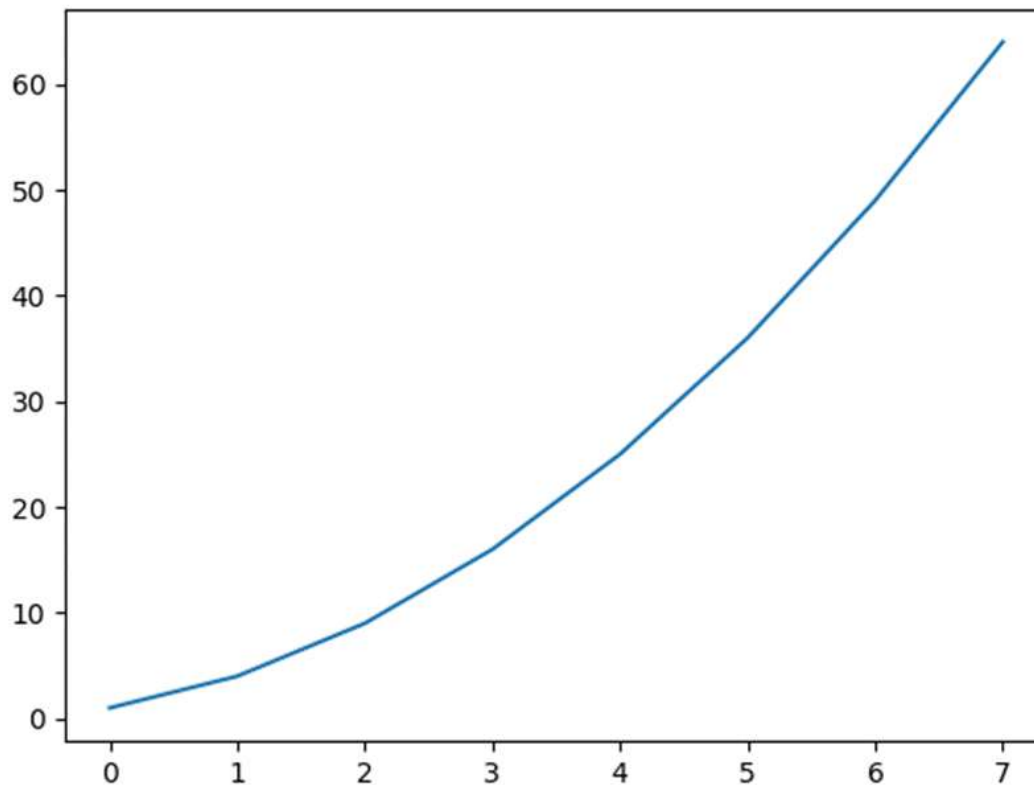
Los gráficos de línea podemos realizarlos a partir de:

- Dos grupos de datos, uno para el eje x y otro para el eje y como vimos en el punto anterior.
- Un grupo de datos para el eje y.

Ejemplo de un solo grupo de datos

Si en lugar de especificar las los datos pertenecientes a las dos coordenadas, presentamos solo un grupo de datos, matplotlib presenta los mismos asignándoles un valor x entero creciente como se puede ver a continuación.

```
y2 = [1,4,9,16,25,36,49,64]  
plt.plot(y2)  
plt.show()
```



Gráficos de dispersión

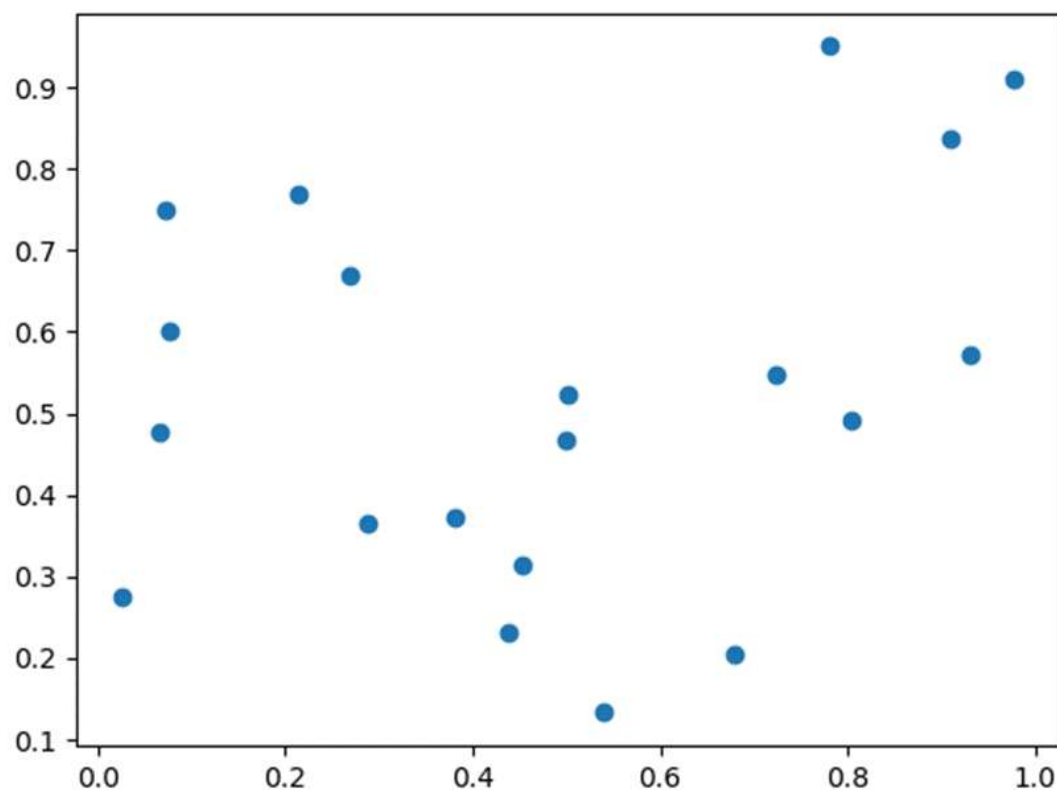
Para generar datos aleatorios se suele partir de un valor llamado “**semilla**”, al igual valor de semilla se obtiene un mismo grupo de datos de dispersión. Para especificar una semilla utilizamos el método de numpy: **numpy.random.seed(valor)**.

A partir de esta semilla podemos crear valores de dispersión mediante el métodos de numpy:

numpy.random.rand(número_de_arrays_generados, elementos_de_la_dispersión)

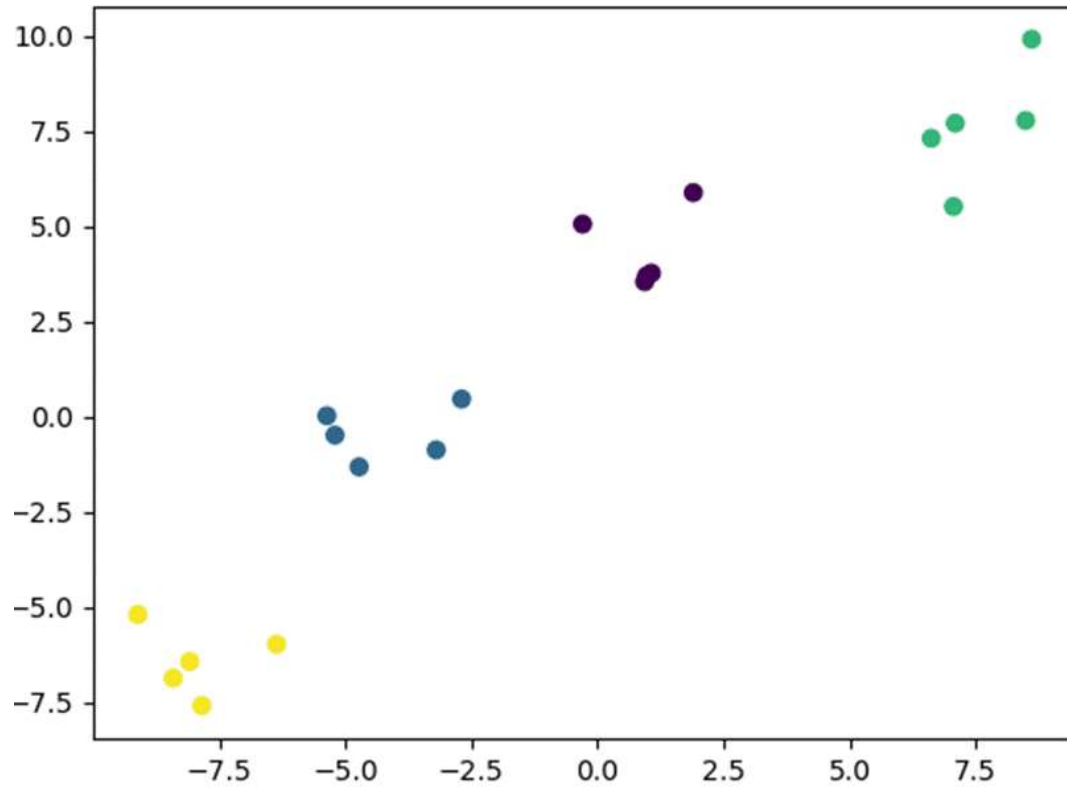
Un ejemplo podría quedar así:

```
import numpy as np
np.random.seed(7)
datos = np.random.rand(2,20)
plt.scatter(datos[0],datos[1])
plt.show()
```



En el caso de que tuviéramos que representar en un mismo gráfico varios grupos de dispersiones, podemos realizarlo con la ayuda del método **make_blobs**, el cual genera puntos basados en la distribución Gaussiana isotrópica según se muestra a continuación:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
datos_mb, features = make_blobs(n_samples=20, centers=4, random_state=3)
plt.scatter(datos_mb[:, 0], datos_mb[:, 1], marker='o', c=features)
plt.show()
```



Aquí los parámetros son:

Parámetro	Descripción
seed	Semilla

n_samples (int)	El número total de puntos divididos en partes iguales entre los grupos. El valor por defecto es 100
centros (int o matriz de forma [n_centers,n_features]):	El número de centros a generar, o las ubicaciones de los centros fijos. El valor por defecto es 3
cluster_std (flotante)	La desviación estándar de los grupos. El valor por defecto es 1.0
shuffle (boolean)	Mezclar las muestras.El valor por defecto es = Verdadero
random_state (int or None)	Semilla utilizada para el generador de números aleatorios, si el valor es None, utiliza el valor de np.random

Polinomios

En ocasiones podemos intentar ajustar una curva por un polinomio, como puede ser en una representación lineal, en este caso podemos utilizar el método **polyfit()** el cual tiene tres parámetros:

- Puntos en el eje x
- Puntos en el eje y
- Grado del polinomio.

El método nos retorna los coeficientes del polinomio, recordemos que la ecuación de un polinomio de grado 'n' es:

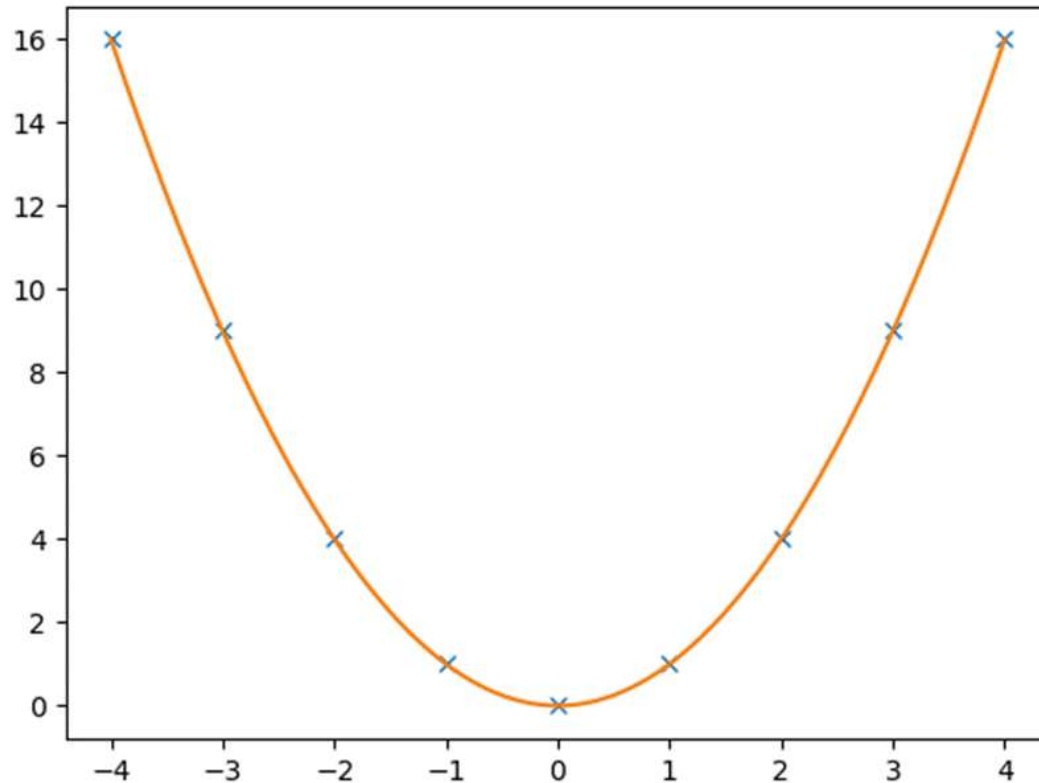
$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0 x^0$$

Los datos retornados son guardados en un array.

Para graficar el polinomio nos valemos del método **poly1d()** como se muestra a continuación en donde se ajustan los puntos por un polinomio de grado 2:

```
x = list(range(-4, 5))
b = np.linspace(-4, 4, 200)
y = [i**2 for i in x]
z = np.polyfit(x, y, 2)

p = np.poly1d(z)
plt.plot(x,y,'x',b,p(b))
```



Mapa de colores

Cuando vamos a utilizar mapas de colores para representar nuestros datos, tenemos que tener en cuenta elegir aquel que mejor represente el conjunto de datos.

Los investigadores han encontrado que el cerebro humano percibe cambios en el parámetro de luminosidad como cambios en los datos mucho mejor que, por ejemplo, cambios en el tono. Por lo tanto, el espectador interpretará mejor los mapas de colores que tienen una luminosidad que aumenta monótonamente a través del mapa de colores.

Los mapas de colores a menudo se dividen en varias categorías según su función, por ejemplo:

- **Secuencial:** Es un cambio en la luminosidad y, a menudo, saturación del color de forma incremental, a menudo con un solo tono; Se debe utilizar para representar información que tiene ordenamiento.
- **Divergente:** Es un cambio en la luminosidad y posiblemente saturación de dos colores diferentes que se encuentran en el medio en un color insaturado; debe usarse cuando la información que se está trazando tiene un valor medio crítico, como la topografía o cuando los datos se desvían alrededor de cero.
- **Cualitativo:** A menudo son colores variados; debe utilizarse para representar información que no tiene orden o relaciones.

Secuencial

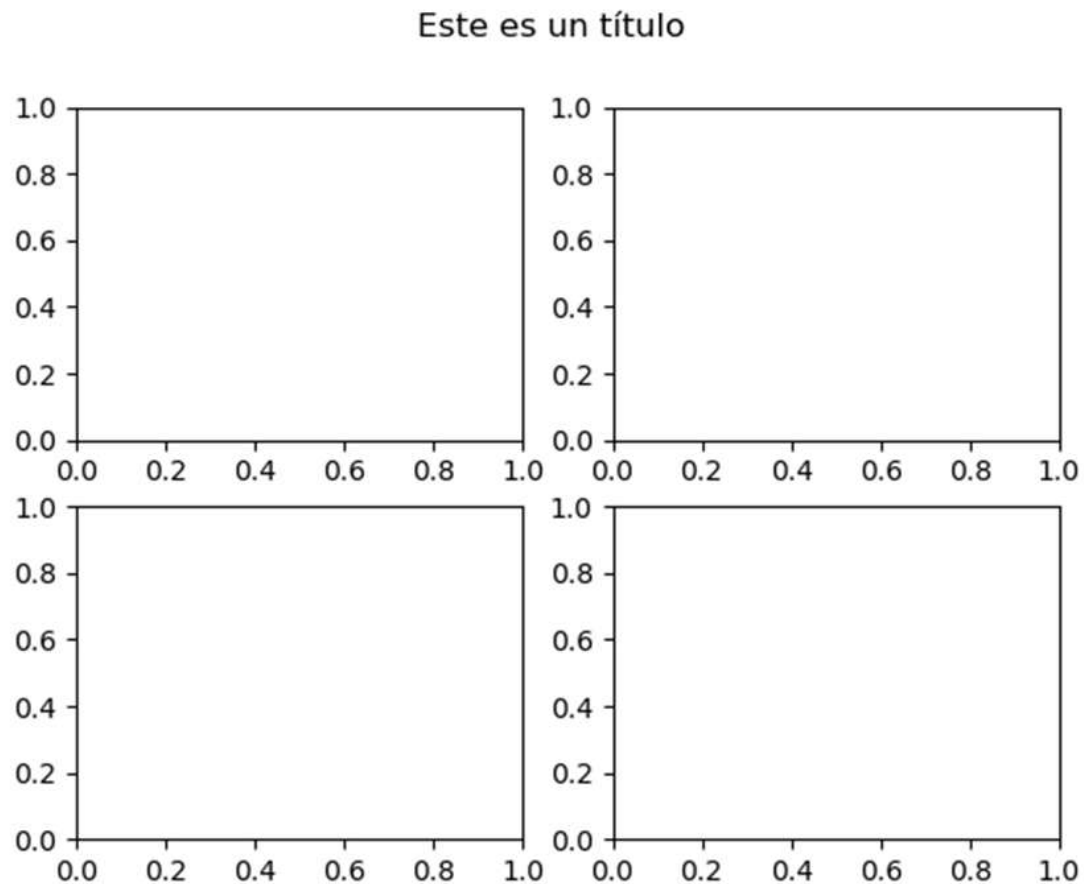
Veamos cómo trabajar con mapas del tipo secuencial, se puede encontrar una referencia completa de todos los tipos de mapas en el siguiente link: <https://matplotlib.org/stable/tutorials/colors/colormaps.html>

Para comprender el siguiente script debemos tener en mente que en matplotlib se denomina:

- **figure:** Al área que contiene la figura completa a representar.
- **axes:** a cada figura representada dentro de figure.
- **axis:** a los ejes de referencia de cada axes.
- **figsize:** es el tamaño de cada axes.

Así por ejemplo si queremos crear cuadro figuras (axes) dentro de un área

```
import matplotlib.pyplot as plt
import numpy as np
fig, axes = plt.subplots(2, 2) # Figura con grillas de ejes 2x2
fig.suptitle('Este es un título')
```



En el siguiente script vamos a utilizar la función **mat_color_gradients()** para generar los listados de gradientes de colores del tipo secuencial que podemos utilizar en matplotlib.

```

import numpy as np
import matplotlib.pyplot as plt

cmaps = [
    ('Sequential', [
        'Greys', 'Purples', 'Blues', 'Greens', 'Oranges', 'Reds',
        'YlOrBr', 'YlOrRd', 'OrRd', 'PuRd', 'RdPu', 'BuPu',
        'GnBu', 'PuBu', 'YlGnBu', 'PuBuGn', 'BuGn', 'YlGn']),
    ('Sequential (2)', [
        'binary', 'gist_yarg', 'gist_gray', 'gray', 'bone', 'pink',
        'spring', 'summer', 'autumn', 'winter', 'cool', 'Wistia',
        'hot', 'afmhot', 'gist_heat', 'copper'])
]

gradient = np.linspace(0, 1, 256)
gradient = np.vstack((gradient, gradient))

def plot_color_gradients(clase_de_mapa, lista_de_datos):
    numero_de_filas = len(lista_de_datos)
    altura = 0.35 + 0.15 + (numero_de_filas + (numero_de_filas-1)*0.1)*0.22
    fig, axes = plt.subplots(nrows=numero_de_filas, figsize=(6.4, altura))
    fig.subplots_adjust(top=1-.35/altura, bottom=.15/altura, left=0.2, right=0.99)

    axes[0].set_title('Mapa de colores de tipo: ' + clase_de_mapa , fontsize=14)

    for ax, nombre in zip(axes, lista_de_datos):
        ax.imshow(gradient, aspect='auto', cmap=plt.get_cmap(nombre))
        ax.text(-.01, .5, nombre, va='center', ha='right', fontsize=10, transform=ax.transAxes)

    for ax in axes:
        ax.set_axis_off()

for clase_de_mapa, lista_de_datos in cmaps:
    plot_color_gradients(clase_de_mapa, lista_de_datos)

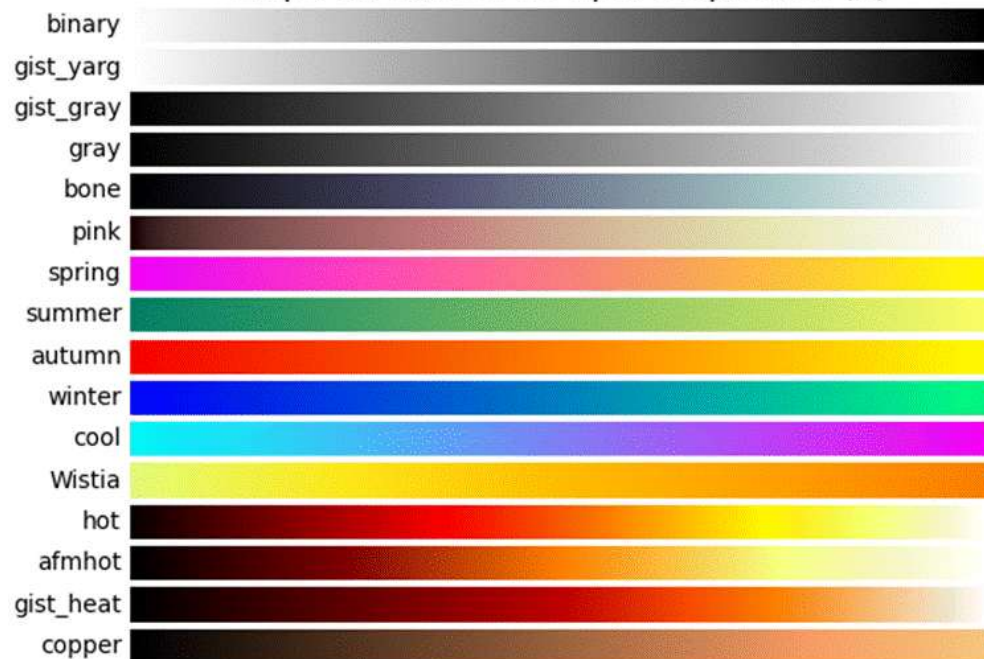
plt.show()

```

Mapa de colores de tipo: Sequential



Mapa de colores de tipo: Sequential (2)

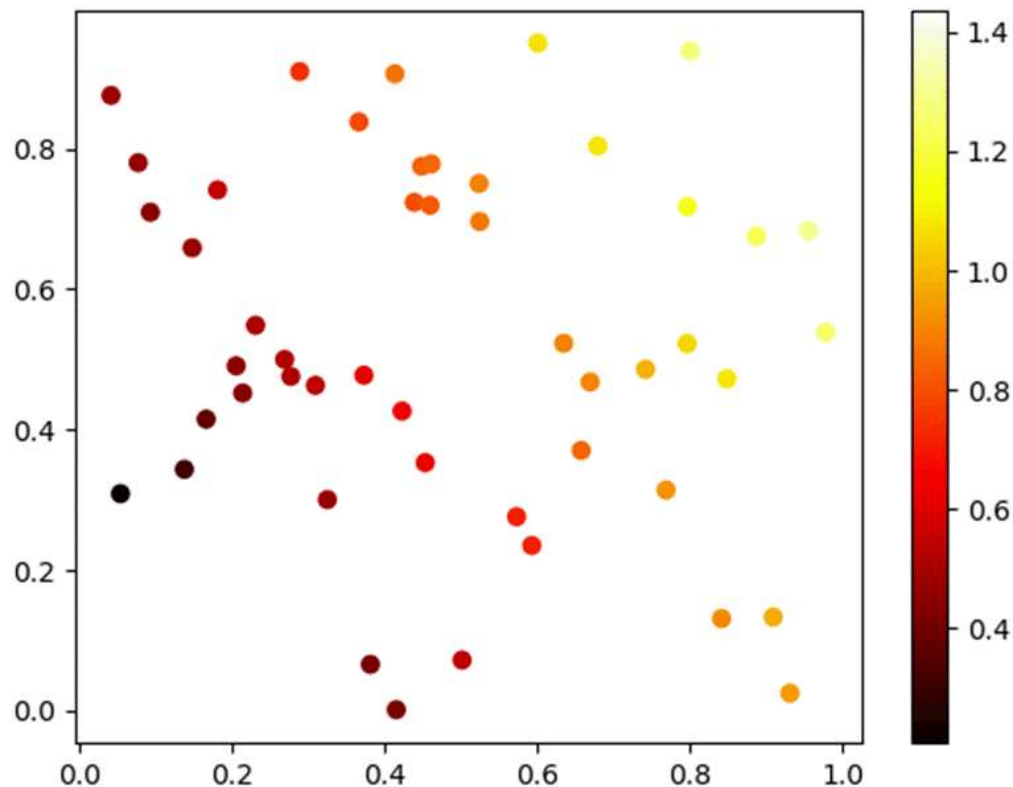


Un ejemplo muy útil de mapa de color, sería asignar colores en una escala según algún criterio específico, por ejemplo en el siguiente script se crea un mapa de valores aleatorios en donde se utiliza el valor en x más el valor en y sobre dos para representar el color de la dispersión.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors

np.random.seed(7)
x,y = zip(*np.random.rand(50,2))
color=[]
print(len(x))
for i in range(0,50):
    color.append(x[i]+y[i]/2)

cmap = matplotlib.colors.LinearSegmentedColormap.from_list("", ["red","yellow","green", "blue"])
plt.scatter(x,y,c=color, cmap='hot')
plt.colorbar()
plt.show()
```

3. Instancia de Evaluación Espacio de entrega de tarea:

En esta instancia de evaluación te pedimos que puedas poner en juego lo que has aprendido hasta este momento. Recordá que siempre podrás volver a la teoría presentada en esta clase o hacer las preguntas pertinentes en los encuentros sincrónicos o las tutorías presenciales.

Es importante tener en cuenta que para aprobar esta instancia deberás:

- Entregar en tiempo y forma
- Realizar un 60% de la actividad correctamente.
- Cumplir con las consignas
- Cumplir con el formato requerido.

La nota mínima para aprobar la tarea es 6.

Ejercicio 1

En el emocionante campo de la Ciencia de Datos, uno de los primeros pasos es la importación y manipulación efectiva de datos. La librería PANDAS en Python es una herramienta esencial para llevar a cabo esta tarea, ya que proporciona funciones y estructuras de datos que permiten la limpieza, transformación y análisis de datos de manera eficiente.

Imaginemos que eres un científico de datos que trabaja en un proyecto de análisis de mercado para una empresa de comercio electrónico. Tu objetivo es recopilar, procesar y analizar datos de ventas que provienen de diferentes fuentes en varios formatos. Para lograrlo, debes diseñar un proceso de importación de datos utilizando PANDAS.

Con alguna herramienta de inteligencia artificial, generá 3 archivos con el siguiente formato para la práctica:

1. **Archivo CSV (Comma-Separated Values):** Tienes un archivo llamado "ventas.csv" que contiene información sobre las ventas de productos. Cada fila representa una venta y contiene varias columnas con los datos sobre las ventas.
2. **Archivo JSON (JavaScript Object Notation):** También tienes un archivo llamado "clientes.json" con datos de los clientes que realizaron las compras. Cada objeto JSON representa un cliente y contiene sus datos.
3. **Hoja de Cálculo Excel:** Existe un archivo llamado "inventario.xlsx" con información sobre el inventario de productos. La hoja de cálculo contiene varias columnas con datos sobre los productos.

Tu tarea es utilizar la librería PANDAS para importar estos datos desde los diferentes formatos (CSV, JSON y Excel), combinarlos en una estructura de datos adecuada y realizar las siguientes acciones:

1. Cargar los datos de ventas desde el archivo CSV y mostrar una vista previa de las primeras filas.
2. Cargar los datos de clientes desde el archivo JSON y mostrar la cantidad total de clientes.
3. Cargar los datos de inventario desde la hoja de cálculo Excel y calcular el promedio de stock disponible para todos los productos.

Al abordar este problema, estarás demostrando tu capacidad para importar, combinar y realizar operaciones básicas de análisis de datos utilizando PANDAS en el contexto del Aprendizaje Automático. ¡Manos a la obra!

El objetivo de este ejercicio es utilizar la librería Matplotlib para realizar una inspección y visualización inicial de un conjunto de datos relacionado con el Aprendizaje Automático. Mediante la creación de diferentes tipos de gráficos, se busca identificar patrones, distribuciones, relaciones y posibles outliers en los datos.

Ejercicio 2

Al descargar el archivo [Automobile.csv](#) se obtiene un conjunto de datos que representa características de automóviles para la predicción de su consumo de combustible. Cada fila del conjunto de datos corresponde a un automóvil y contiene las siguientes columnas:

- `mpg`: Millas por galón (consumo de combustible)
- `cylinders`: Cilindros en el motor
- `displacement`: Desplazamiento del motor (en pulgadas cúbicas)
- `horsepower`: Potencia del motor (caballos de fuerza)
- `weight`: Peso del automóvil
- `acceleration`: Aceleración (0-60 mph en segundos)
- `model\_year`: Año del modelo (ej. 70 representa 1970)
- `origin`: Origen del automóvil (1: Estados Unidos, 2: Europa, 3: Asia)

Tareas a realizar:

1. Cargar el conjunto de datos en una estructura adecuada para su manipulación.
2. Crear un gráfico de dispersión para visualizar las relaciones entre los pares de características numéricas `mpg` y `weight`.
3. Generar un histograma del consumo de combustible (`mpg`) para analizar su distribución.
4. Graficar un diagrama de caja (boxplot) de las características `cylinders` y `origin` para identificar posibles valores atípicos.
5. Realizar un gráfico de barras que muestre la distribución de automóviles según su año de modelo (`model\_year`) y origen (`origin`).

Recuerda que la visualización de datos es una etapa esencial en el proceso de Aprendizaje Automático, ya que proporciona intuiciones valiosas que pueden guiar decisiones posteriores en la construcción y evaluación de modelos predictivos. Utiliza Matplotlib para crear gráficos claros y etiquetados que ayuden en el análisis de los datos.

En el próximo encuentro sincrónico haremos intercambio de ideas sobre cómo resolver esta actividad.

5. Cierre:

Sin duda, tener conocimiento en el uso de herramientas como la librería Pandas y habilidades en la importación y manipulación de datos es esencial en el ámbito de la Ciencia de Datos y la Inteligencia Artificial. Aquí hay algunas reflexiones sobre la importancia de este conocimiento:

- *Base Fundamental:* La importación y manipulación de datos constituyen la base fundamental para cualquier análisis o modelado posterior en Ciencia de Datos. Los datos en su forma cruda pueden ser desorganizados y poco estructurados. Saber cómo limpiar, transformar y combinar estos datos es crucial para obtener información útil y significativa.
- *Toma de Decisiones Informadas:* En un mundo impulsado por datos, las decisiones informadas son vitales. La habilidad de importar y manipular datos te permite obtener insights valiosos y tomar decisiones basadas en evidencias, lo que puede tener un impacto significativo en la estrategia de una empresa o en la investigación científica.
- *Eficiencia y Productividad:* La manipulación manual de grandes conjuntos de datos es ineficiente y propenso a errores. Las herramientas como Pandas automatizan y agilizan el proceso, lo que aumenta la productividad y permite a los profesionales de Ciencia de Datos concentrarse en tareas más analíticas y creativas.
- *Preparación de Datos para Modelado:* Antes de construir modelos de Machine Learning o realizar análisis estadísticos, los datos deben estar en un formato adecuado. La habilidad de transformar datos en formatos que puedan ser procesados por algoritmos de IA es esencial para crear modelos precisos y útiles.
- *Exploración y Visualización:* La manipulación de datos también es crucial para la exploración inicial de los mismos. Comprender las características de los datos a través de gráficos y estadísticas descriptivas puede ayudar a identificar patrones, anomalías y tendencias que luego pueden ser abordados en análisis más profundos.
- *Colaboración Interdisciplinaria:* La Ciencia de Datos e Inteligencia Artificial a menudo involucran equipos interdisciplinarios. Tener conocimiento en la manipulación de datos te permite comunicarte de manera efectiva con expertos en dominios específicos y trabajar juntos en la interpretación y análisis de datos.
- *Flexibilidad y Adaptabilidad:* Los datos pueden provenir de diversas fuentes y en diferentes formatos. Saber cómo importar y unificar estos datos te brinda la flexibilidad para trabajar con una amplia gama de datos y abordar problemas variados.

El conocimiento en la importación y manipulación de datos con herramientas como PANDAS no solo es una habilidad técnica, sino que es la columna vertebral de cualquier proyecto de Ciencia de Datos o Inteligencia Artificial exitoso. Facilita la transformación de datos en conocimiento, permitiendo la toma de decisiones fundamentadas y el desarrollo de modelos y soluciones avanzadas.

Por otro lado, el dominio de la librería Matplotlib para la inspección y visualización de datos en el contexto del Aprendizaje Automático es fundamental por diversas razones. En primer lugar, la representación gráfica de datos permite una comprensión más intuitiva y profunda de los patrones y relaciones presentes en los conjuntos de datos, lo que resulta esencial para la toma de decisiones informadas en cualquier proceso de análisis. Además, la habilidad para crear visualizaciones claras y efectivas es crucial al comunicar resultados a colegas, superiores o audiencias no técnicas, ya que una imagen bien diseñada puede transmitir información de manera más impactante y accesible que las

descripciones verbales o tablas de números. En el ámbito del Aprendizaje Automático, la capacidad de visualizar datos también es esencial para detectar anomalías, entender la distribución de variables, seleccionar características relevantes y evaluar el rendimiento de modelos predictivos. En última instancia, el conocimiento de Matplotlib y las técnicas de visualización asociadas potencian la capacidad de explorar, analizar y presentar datos de manera efectiva, brindando una ventaja invaluable para tomar decisiones informadas y desarrollar soluciones precisas en problemas de Aprendizaje Automático.

Te esperamos en los próximos encuentros.

Para finalizar te dejamos estos recursos que pueden ayudarte a entender el tema un poco más.

Sitios

- *Requests: HTTP para Humanos — documentación de Requests - 1.1.0*, <http://es.python-requests.org/es/latest/>. Obtenido 11 Agosto 2023.
- *Jupyter and the future of IPython — IPython*, <http://ipython.org/>. Obtenido 11 Agosto 2023.
- *pandas - Python Data Analysis Library*, <https://pandas.pydata.org/>. Obtenido 11 Agosto 2023.
- "Installation — Anaconda documentation." *Anaconda documentation*, <https://docs.anaconda.com/free/anaconda/install/>. Obtenido 11 Agosto 2023.
- "Matplotlib." *Matplotlib — Visualization with Python*, <https://matplotlib.org/>. Accedido el 23 Junio del 2023.
- "NumPy.org." *NumPy.org*, <http://www.numpy.org/>. Accedido el 23 Junio del 2023.
- "pandas." *pandas - Python Data Analysis Library*, <https://pandas.pydata.org/>. Accedido el 23 Junio del 2023.

6. Bibliografía Obligatoria:

- RUSSELL, R. (2018): Machine Learning. Guia Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python
- CANE, A. (2019): Programación Con Python. Guía completa para principiantes.
- HAN, R. (2019): Matemáticas del Aprendizaje Automático. Introducción a la analítica de datos e Inteligencia Artificial.

- Mark Lutz, O'Reilly (2013, 02). Programming Python 5th Edition By
- Yuxi (Hayden) Liu, (2019, 02). Python Machine Learning By Example, Second Edition
- Alexander Combs, Michael Roman, (2019, 02). Python Machine Learning Blueprints, Second Edition